

RELATÓRIO TÉCNICO

PARTE PRÁTICA

Entrega Contínua de uma Plataforma de Pedidos em Microserviços
Do Docker Compose ao Kubernetes com Observabilidade e CI/CD

Loja Veloz · Cloud DevOps Engineer · 2025

Docker

K8s

CI/CD

Terraform

Prometheus

Grafana

Jaeger

HPA

1. Visão Geral da Arquitetura Prática

A plataforma **Pedidos Veloz** é composta por cinco serviços principais organizados em camadas de responsabilidade distintas. A solução cobre o ciclo completo: desenvolvimento local padronizado com Docker Compose, containerização com boas práticas de segurança, orquestração Kubernetes para produção, pipeline CI/CD automatizado e observabilidade com métricas, logs e tracing distribuído.

- API Gateway → Serviço de Pedidos → Serviço de Pagamentos
- Serviço de Pedidos → Broker (RabbitMQ) → Serviço de Estoque
- Todos os serviços → PostgreSQL (fase inicial, com evolução para DB por domínio)
- Observabilidade: Prometheus · Grafana · Jaeger (via OpenTelemetry)

1.1 Repositório e Estrutura de Pastas

```
# Estrutura do Repositório
pedidos-veloz/
├── docker-compose.yml      # Ambiente local completo
├── .env.example            # Variáveis de ambiente de exemplo
├── services/
│   ├── api-gateway/       # Dockerfile + código
│   ├── pedidos/           # Dockerfile + código
│   ├── pagamentos/       # Dockerfile + código
│   └── estoque/           # Dockerfile + código
├── k8s/
│   ├── base/              # Deployments, Services, ConfigMaps, Secrets
│   ├── overlays/prod/     # Kustomize patches de produção
│   └── monitoring/        # Prometheus, Grafana, Jaeger manifests
├── .github/workflows/     # Pipelines GitHub Actions
└── ci.yml                 # Build, test, scan, push
```

```
■ ■■■ cd.yml # Deploy para Kubernetes
■ ■■■ terraform/ # IaC - cluster, VPC, registry
■ ■■■ main.tf
■ ■■■ variables.tf
■ ■■■ modules/
■ ■■■ README.md
```

2. Ambiente Local com Docker Compose

O ambiente local é 100% reproduzível. Qualquer desenvolvedor sobe todos os serviços com um único comando, eliminando inconsistências entre máquinas e garantindo paridade com o ambiente de produção (princípio 10 do 12-Factor App).

2.1 Inicialização

```
# Comandos de execução local
# Copiar variáveis de ambiente
cp .env.example .env

# Subir todos os serviços (build + start)
docker compose up --build -d

# Verificar status
docker compose ps

# Logs em tempo real
docker compose logs -f pedidos

# Encerrar e remover volumes
docker compose down -v
```

2.2 Trecho do docker-compose.yml

```
# docker-compose.yml (resumido)
version: '3.9'
services:
  api-gateway:
    build: ./services/api-gateway
    ports: ['8080:8080']
    environment:
      - PEDIDOS_URL=http://pedidos:3001
    networks: [app-net]
    depends_on: [pedidos]
  pedidos:
    build: ./services/pedidos
    environment:
      - DATABASE_URL=postgres://user:${DB_PASS}@postgres:5432/pedidos
      - RABBITMQ_URL=amqp://rabbitmq:5672
```

```
networks: [app-net]
depends_on: [postgres, rabbitmq]

postgres:
  image: postgres:16-alpine
  volumes: [pg-data:/var/lib/postgresql/data]
  environment: {POSTGRES_PASSWORD: '${DB_PASS}'}
  networks: [app-net]

rabbitmq:
  image: rabbitmq:3.13-management-alpine
  ports: ['15672:15672']
  networks: [app-net]

networks:
  app-net: {driver: bridge}

volumes:
  pg-data:
```

3. Containerização e Versionamento

3.1 Dockerfile Multi-Stage – Serviço de Pedidos (Node.js)

A construção em múltiplos estágios separa o ambiente de build do artefato final, resultando em imagens até 70% menores e sem dependências de desenvolvimento em produção.

[illegible]

3.2 Versionamento e Publicação de Imagens

```
# Versionamento de imagens

# Tag semântica gerada automaticamente pelo CI
IMAGE=ghcr.io/loja-veloz/pedidos
TAG=${GITHUB_REF_NAME}-${GITHUB_SHA::7} # ex: v1.3.0-a4f2c1d

docker build -t $IMAGE:$TAG -t $IMAGE:latest ./services/pedidos
docker push $IMAGE:$TAG
docker push $IMAGE:latest
```

■ Boas práticas aplicadas:

- Usuário não-root em todos os containers
- Imagens base Alpine/Distroless (superfície mínima de ataque)
- Multi-stage build: imagem final sem compiladores ou ferramentas de dev
- .dockerignore exclui node_modules, .git, *.test.js, .env
- HEALTHCHECK definido em todos os Dockerfiles

4. Kubernetes – Produção Mínima

4.1 Deployment – Serviço de Pedidos

```
# k8s/base/deployment-pedidos.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pedidos
  namespace: loja-veloz
spec:
  replicas: 3
  selector:
    matchLabels: {app: pedidos}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: pedidos}
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile: {type: RuntimeDefault}
      containers:
```

```

- name: pedidos
  image: ghcr.io/loja-veloz/pedidos:v1.3.0
  ports: [{containerPort: 3001}]
  envFrom:
    - configMapRef: {name: pedidos-config}
    - secretRef:     {name: pedidos-secrets}
  resources:
    requests: {cpu: 100m, memory: 128Mi}
    limits:   {cpu: 500m, memory: 512Mi}
  readinessProbe:
    httpGet: {path: /health, port: 3001}
    initialDelaySeconds: 10
    periodSeconds: 5
  livenessProbe:
    httpGet: {path: /health, port: 3001}
    initialDelaySeconds: 30
    periodSeconds: 15

```

4.2 ConfigMap e Secret

```

# ConfigMap e Secret
# ConfigMap - configurações não sensíveis
apiVersion: v1
kind: ConfigMap
metadata:
  name: pedidos-config
  namespace: loja-veloz
data:
  NODE_ENV: 'production'
  RABBITMQ_URL: 'amqp://rabbitmq-svc:5672'
  LOG_LEVEL: 'info'

---
# Secret - credenciais (base64 ou integrado a Vault)
apiVersion: v1
kind: Secret
metadata:
  name: pedidos-secrets
  namespace: loja-veloz
type: Opaque
data:
  DATABASE_URL: <base64> # gerenciado via External Secrets Operator
  JWT_SECRET:   <base64>

```

4.3 Horizontal Pod Autoscaler (HPA)

```
# HPA - Serviço de Pedidos
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: pedidos-hpa
  namespace: loja-veloz
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pedidos
  minReplicas: 3
  maxReplicas: 15
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 60}
    - type: Resource
      resource:
        name: memory
        target: {type: Utilization, averageUtilization: 70}
```

Justificativa do HPA: O Serviço de Pedidos é o principal gargalo durante picos de tráfego. Com target de 60% de CPU, o HPA escala preventivamente antes de atingir o limite, garantindo headroom para absorver spikes. O mínimo de 3 réplicas assegura disponibilidade mesmo durante updates. Para a campanha promocional, o minReplicas pode ser elevado manualmente para 8.

4.4 Segurança

- Pod Security Admission (perfil 'restricted'): bloqueia privilégios, runAsRoot e hostPath em todos os pods do namespace loja-veloz.
- Network Policies: serviços só recebem tráfego de origens autorizadas (ex.: Pedidos só aceita conexões do API Gateway e do RabbitMQ).
- RBAC: ServiceAccounts por serviço com permissões mínimas (least privilege).
- External Secrets Operator integra o K8s ao HashiCorp Vault para injeção segura de credenciais sem expor valores em manifests.
- Scan de imagens no CI com Trivy: falhas críticas bloqueiam o pipeline.

5. Pipeline CI/CD – GitHub Actions

O pipeline é dividido em dois workflows: **ci.yml** (acionado em todo push/PR) e **cd.yml** (acionado após merge na branch main). Cada serviço possui seu próprio pipeline, permitindo releases independentes.

Estágio	Ferramenta	Descrição
Lint & Format	ESLint / Prettier	Valida estilo e qualidade de código
Testes unitários	Jest / PyTest	Cobertura mínima de 80% exigida
Build da imagem	Docker buildx	Build multi-arch com cache de camadas
Scan de segurança	Trivy	Bloqueia CVEs críticas e altas
Push para registry	GHCR	Tag semântica: versão + SHA curto
Deploy em K8s	kubectl / Helm	Rolling update com rollback automático
Smoke test	curl / k6	Valida endpoints após deploy

Tabela 2 – Estágios do Pipeline CI/CD

5.1 Trecho do Workflow CI (ci.yml)

```
# ci.yml (simplificado)
name: CI Pipeline
on:
  push: {branches: [main, develop]}
  pull_request: {branches: [main]}

jobs:
  test-and-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with: {node-version: '20', cache: 'npm'}

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Unit tests
        run: npm test -- --coverage

      - name: Build Docker image
        run: |
          docker build -t ghcr.io/loja-veloz/pedidos:${{ github.sha }} .

      - name: Security scan (Trivy)
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'ghcr.io/loja-veloz/pedidos:${{ github.sha }}'
          exit-code: '1'
```

```

    severity: 'CRITICAL,HIGH'

  - name: Push image
    if: github.ref == 'refs/heads/main'
    run: |
      echo ${ secrets.GITHUB_TOKEN } | docker login ghcr.io -u ${ github.actor } --password-stdin
      docker push ghcr.io/loja-veloz/pedidos:${ github.sha }

```

6. Observabilidade, Deploy e Escalabilidade

6.1 Stack de Observabilidade

A observabilidade da plataforma é implementada sobre os três pilares: métricas, logs e traces, com ferramentas cloud-native integradas via Kubernetes.

Pilar	Ferramenta	Função
Métricas	Prometheus + Grafana	Coleta de métricas de sistema e negócio; dashboards e alertas
Logs	Fluentd + Loki	Agregação de logs estruturados (JSON) com busca e correlação
Traces	OpenTelemetry + Jaeger	Rastreamento distribuído end-to-end entre todos os serviços
Alertas	Alertmanager	Notificações via Slack/PagerDuty para SLO violations

Tabela 3 – Stack de Observabilidade

6.2 Métricas Chave de Negócio (SLIs/SLOs)

- SLO de Disponibilidade: $\geq 99,5\%$ de uptime mensal para o endpoint /pedidos
- SLO de Latência: p95 de /pedidos/criar $< 300\text{ms}$
- SLO de Taxa de Erro: $< 0,5\%$ de respostas 5xx no API Gateway
- Métrica de negócio: Pedidos processados por minuto (alerta se cair $> 20\%$)

6.3 Tracing Distribuído com OpenTelemetry

```

# Instrumentação OpenTelemetry - pedidos/src/tracing.ts
# Instrumentação no Serviço de Pedidos (Node.js)
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT, // http://jaeger:4318
  }),

```



```

    instrumentations: [getNodeAutoInstrumentations()],
    serviceName: 'pedidos-service',
  });
  sdk.start();

```

6.4 Estratégia de Deploy: Rolling Update com Gate de Qualidade

A estratégia escolhida é **Rolling Update**, nativa do Kubernetes, configurada com `maxUnavailable: 0` e `maxSurge: 1`. Isso garante que sempre há réplicas saudáveis servindo tráfego durante a atualização. O rollback automático é acionado se os probes de readiness falharem por mais de 5 minutos.

```

# Estratégia de deploy com rollback
# Deploy via CD pipeline com validação de rollout
kubectl set image deployment/pedidos pedidos=ghcr.io/loja-veloz/pedidos:$TAG -n loja-veloz
kubectl rollout status deployment/pedidos -n loja-veloz --timeout=5m

# Rollback automático em caso de falha
if [ $? -ne 0 ]; then
    kubectl rollout undo deployment/pedidos -n loja-veloz
    echo '■ Deploy falhou - rollback executado'
    exit 1
fi
echo '■ Deploy concluído com sucesso'

```

Evolução futura: Canary release com Argo Rollouts permitirá direcionar 5% do tráfego para a nova versão e promover gradualmente com base em métricas de erro e latência — especialmente importante para a campanha promocional.

7. Infraestrutura como Código – Terraform

Toda a infraestrutura cloud é provisionada via Terraform, garantindo reprodutibilidade, auditoria via Git e eliminação de configurações manuais.

7.1 Módulos Terraform

Módulo	Recursos Provisionados
modules/network	VPC, subnets públicas/privadas, NAT Gateway, firewall rules
modules/gke	Cluster GKE Autopilot, node pools, RBAC, Workload Identity
modules/registry	Google Artifact Registry para imagens Docker versionadas
modules/database	Cloud SQL (PostgreSQL 16), backups, high availability
modules/monitoring	Workspace do Google Cloud Monitoring, alertas base

Tabela 4 – Módulos Terraform

```

# terraform/main.tf
# terraform/main.tf (esqueleto)
terraform {

```

```

required_providers {
  google = { source = 'hashicorp/google', version = '~> 5.0' }
}
backend 'gcs' {
  bucket = 'loja-veloz-tfstate'
  prefix = 'prod/state'
}
}

module 'network' {
  source      = './modules/network'
  project_id  = var.project_id
  region     = var.region
}

module 'gke' {
  source      = './modules/gke'
  project_id  = var.project_id
  network_id  = module.network.vpc_id
  min_nodes   = 2
  max_nodes   = 10
}

```

8. Case de Referência – Google Microservices Demo

O projeto de referência adotado é o **microservices-demo** do Google (github.com/GoogleCloudPlatform/microservices-demo), uma loja online de exemplo com 11 microsserviços em diferentes linguagens, implantada no Google Kubernetes Engine. Este case público demonstra exatamente o padrão arquitetural adotado: Docker Compose para desenvolvimento local, manifestos Kubernetes para produção, OpenTelemetry para tracing distribuído, Skaffold para CI/CD e HPA para escalonamento automático.

- Repositório: github.com/GoogleCloudPlatform/microservices-demo
- Padrões validados: multi-stage Dockerfiles, ConfigMaps/Secrets, HPA, readiness/liveness probes, OpenTelemetry end-to-end
- Lições aplicadas à Loja Veloz: separação de responsabilidades por serviço, comunicação assíncrona via broker, observabilidade integrada desde o início

9. Plano de Execução – MVP em 4 Semanas

Semana	Foco	Entregáveis
Semana 1	Fundação	Docker Compose funcional • Dockerfiles otimizados • Repositório estruturado • README

Semana 2	Kubernetes	Manifests K8s (Deployments, Services, ConfigMaps, Secrets, HPA) • Cluster de staging
Semana 3	CI/CD + Segurança	Pipeline GitHub Actions completo • Trivy scan • Deploy automatizado • Terraform skeleton
Semana 4	Observabilidade + Campanha	Prometheus, Grafana, Jaeger • SLOs definidos • Load test (k6) • Pre-scaling para campanha

Tabela 5 – Cronograma de 4 semanas

10. Referências

CNCF. *Kubernetes Documentation*. Disponível em: <https://kubernetes.io/docs>. Acesso em: 2025.

Docker Inc. *Docker Compose Documentation*. Disponível em: <https://docs.docker.com/compose>. Acesso em: 2025.

GitHub. *GitHub Actions Documentation*. Disponível em: <https://docs.github.com/actions>. Acesso em: 2025.

HashiCorp. *Terraform Documentation*. Disponível em: <https://developer.hashicorp.com/terraform>. Acesso em: 2025.

OpenTelemetry. *OpenTelemetry Documentation*. Disponível em: <https://opentelemetry.io/docs>. Acesso em: 2025.

Google Cloud. *Microservices Demo – Reference Architecture*. Disponível em: <https://github.com/GoogleCloudPlatform/microservices-demo>. Acesso em: 2025.

Aqua Security. *Trivy – Container Vulnerability Scanner*. Disponível em: <https://github.com/aquasecurity/trivy>. Acesso em: 2025.

HUMBLE, J.; FARLEY, D. *Continuous Delivery*. Addison-Wesley, 2010.

NEWMAN, S. *Building Microservices*. O'Reilly Media, 2nd ed., 2021.

Prometheus. *Prometheus Documentation*. Disponível em: <https://prometheus.io/docs>. Acesso em: 2025.